TALLER FORMATIVO GUIADO: ORQUESTACIÓN DE CONTENEDORES Y ALTA DISPONIBILIDAD

INGENIERÍA DE SOFTWARE V (CLÚSTER KUBERNETES MULTI-NODO)

FACULTAD DE INGENIERÍA, DISEÑO Y CIENCIAS APLICADAS INTELIGENTES

DEPARTAMENTO DE COMPUTACIÓN Y SISTEMAS

INFORMACIÓN DE LA ACTIVIDAD

IDENTIFICACIÓN DEL EQUIPO

CÓDIGO:	09776 - TIC	**PERÍODO:**	2026-2
NRC / GRUPO:	11005 / Grupo 001	**PONDERACIÓN:**	Formativa Guiada (Laboratorio U6)
MODALIDAD:	Equipos de 7 Estudiantes	**DURACIÓN:**	3 Horas Lab + Cierre Autónomo
POLÍTICA IAG:	Nivel 3: Colaboración Asistida y Auditoría	**CALIFICACIÓN:**	_____ / 5.0
GRUPO Nº:	_____	**Nº MÁQUINAS:**	**7 Computadores Dedicados**
NODO 1 (Master):	_____	**CÓDIGO / ESTUDIANTE:**	_____
NODO 2 (Worker 1):	_____	**CÓDIGO / ESTUDIANTE:**	_____
NODO 3 (Worker 2):	_____	**CÓDIGO / ESTUDIANTE:**	_____
NODO 4 (Worker 3):	_____	**CÓDIGO / ESTUDIANTE:**	_____
NODO 5 (Worker 4):	_____	**CÓDIGO / ESTUDIANTE:**	_____
NODO 6 (Ingress):	_____	**CÓDIGO / ESTUDIANTE:**	_____
NODO 7 (Bastion):	_____	**CÓDIGO / ESTUDIANTE:**	_____

1 Contexto de Ingeniería y Arquitectura del Clúster de 7 Computadores

En entornos industriales de software en la nube, las arquitecturas basadas en contenedores no se ejecutan sobre servidores individuales aislados. Si un host físico colapsa por una falla de hardware, corte eléctrico o saturación de recursos, todos los contenedores alojados en él mueren sin posibilidad de auto-recuperación. Para superar esta limitación crítica, la industria adopta **Kubernetes (K8s)** como el estándar de facto para la orquestación distribuida, el autoescalado y la resiliencia en clúster.

En este taller evaluativo, cada equipo de trabajo dispone de **7 computadores dedicados en el laboratorio (conectados en la red LAN o VPN ZeroTier)**. El equipo diseñará, aprovisionará, configurará y operará un **Clúster Kubernetes Multi-Nodo real**, distribuyendo responsabilidades de cómputo, almacenamiento, enrutamiento y auditoría de resiliencia mediante Chaos Engineering.

Enfoque de Seguridad Docente (Docker-Native / Sin Sudo): Para garantizar la seguridad del laboratorio universitario y evitar la alteración de archivos sensibles del sistema operativo (`/etc/fstab`, parámetros de red o elevación de privilegios root), **los estudiantes operan exclusivamente con el usuario** `docker`. El clúster se despliega utilizando la imagen oficial de Kubernetes `rancher/k3s` ejecutada en contenedores Docker con los modificadores `--privileged --net=host`. Esto permite:

- Operar al 100 % sin requerir permisos de `sudo` ni contraseñas administrativas.
- Desacoplar el swap del host mediante la bandera del Kubelet `--kubelet-arg="fail-swap-on=false"`.
- Compartir de forma transparente la interfaz física de red entre los 7 equipos (`--net=host`), exponiendo directamente el API Server (6443), Kubelet (10250), Flannel (8472 UDP) e Ingress (80/443).
- Limpiar completamente la máquina en 5 segundos al finalizar la práctica.

1.1 Topología del Clúster y Asignación de Nodos

Máquina	Hostname Sugerido	Rol en el Clúster	Cargas de Trabajo Asignadas
PC 1	`k8s-control-plane`	Control Plane (Master)	`kube-apiserver`, `etcd`, `scheduler`, `controller-manager`.
PC 2	`k8s-worker-backend-1`	Worker Node 1	Microservicios Backend (Réplicas A), Runtime containerd.
PC 3	`k8s-worker-backend-2`	Worker Node 2	Microservicios Backend (Réplicas B - Alta Disponibilidad).
PC 4	`k8s-worker-frontend`	Worker Node 3	Capa de Presentación Web (Single Page Application / Nginx).
PC 5	`k8s-worker-data`	Worker Node 4	Base de Datos (PostgreSQL/Redis), Volúmenes Persistentes.
PC 6	`k8s-worker-ingress`	Worker Node 5 (Edge)	NGINX Ingress Controller, Balanceo L7, Enrutamiento por Host/Path.
PC 7	`k8s-bastion-ops`	Nodo Bastion / SRE Ops	Cliente externo, generador de carga HTTP, inyector de caos y fallos.

2 Procedimiento Experimental y Fases de Implementación

2.1 Fase 1: Aprovisionamiento de Red y Ensamblaje del Clúster (7 Computadores)

Tarea 1.1: Preparación del Espacio de Trabajo en los 7 Nodos (Sin Sudo). En cada uno de los 7 computadores (PC 1 a PC 7), verificar el acceso a Docker y la dirección IP asignada:

1. Identificar la dirección IP de la máquina en la red local o ZeroTier:

```
hostname -I | awk '{print $1}'
```

2. Verificar que el usuario actual tenga acceso operativo al socket de Docker:

```
docker ps
```

3. Crear el directorio de trabajo del laboratorio en el home del estudiante:

```
mkdir -p ~/k8s-cluster && cd ~/k8s-cluster
```

4. **Nota de Swap:** Gracias al desacople en contenedor con `fail-swap-on=false`, **NO** es necesario ejecutar `swapoff -a` ni modificar `/etc/fstab`.

Tarea 1.2: Inicialización del Control Plane en Contenedor (PC 1). En la máquina **PC 1**, el Estudiante 1 inicializa el plano de control ejecutando el contenedor oficial de K3s con red de host y permisos de contenedor:

```
# 1. Crear carpeta local para extraer el kubeconfig con permisos de usuario
mkdir -p ~/k8s-cluster/k8s-data && cd ~/k8s-cluster

# 2. Levantar el nodo Master vinculando la IP de PC 1
docker run -d \
  --name k8s-master \
  --restart unless-stopped \
```

```
  --privileged \
  --net=host \
  -e K3S_TOKEN=icesi-k8s-cluster-grupo01 \
  -e K3S_KUBECONFIG_OUTPUT=/output/kubeconfig.yaml \
  -e K3S_KUBECONFIG_MODE=666 \
  -v $(pwd)/k8s-data:/output \
  rancher/k3s:v1.31.1-k3s1 server \
  --bind-address <IP_LAN_PC1> \
  --node-ip <IP_LAN_PC1> \
  --kubelet-arg="fail-swap-on=false"

# 3. Configurar kubectl localmente para el usuario (sin sudo)
export KUBECONFIG=$(pwd)/k8s-data/kubeconfig.yaml
sed -i "s/127.0.0.1/<IP_LAN_PC1>/g" $(pwd)/k8s-data/kubeconfig.yaml
echo "export KUBECONFIG=$(pwd)/k8s-data/kubeconfig.yaml" >> ~/.bashrc
```

Verificar que el API Server responda: `kubectl get nodes`. Compartir el archivo `kubeconfig.yaml` y el token `icesi-k8s-cluster-grupo01` con los compañeros del grupo.

Tarea 1.3: Unión de los Worker Nodes mediante Contenedores (PC 2 a PC 6). En cada una de las máquinas asignadas como Workers (PC 2, PC 3, PC 4, PC 5 y PC 6), el estudiante respectivo ejecuta el agente de K3s apuntando a la IP de PC 1:

```
# En PC 2, 3, 4, 5 y 6 (remplazar <IP_LAN_PC1> y <IP_DE_ESTE_PC>):
docker run -d \
  --name k8s-worker \
  --restart unless-stopped \
  --privileged \
  --net=host \
  -e K3S_URL=https://<IP_LAN_PC1>:6443 \
  -e K3S_TOKEN=icesi-k8s-cluster-grupo01 \
  rancher/k3s:v1.31.1-k3s1 agent \
  --node-ip <IP_DE_ESTE_PC> \
  --kubelet-arg="fail-swap-on=false"
```

Tarea 1.4: Validación y Etiquetado de Nodos. Desde el Control Plane (PC 1) o desde el Bastion (PC 7 configurado con el `kubeconfig`), ejecutar:

```
kubectl get nodes -o wide
```

Criterio de Aceptación Obligatorio: Los 6 nodos del clúster deben figurar en estado `Ready`. A continuación, aplicar etiquetas semánticas a los nodos para la planificación de cargas de trabajo (utilizando los nombres de host reportados en la columna `NAME`):

```
kubectl label node <HOSTNAME_PC2> tier=backend zone=primary
kubectl label node <HOSTNAME_PC3> tier=backend zone=secondary
kubectl label node <HOSTNAME_PC4> tier=frontend
kubectl label node <HOSTNAME_PC5> tier=database
kubectl label node <HOSTNAME_PC6> role=ingress-gateway
```

2.2 Fase 2: Despliegue Declarativo de la Aplicación Multi-Capa con Anti-Afinidad

El equipo desplegará una solución compuesta por tres capas: Base de Datos/Almacenamiento (PC 5), Backend REST API (PC 2 y PC 3) y Frontend Web (PC 4).

Tarea 2.1: Gestión de Configuración y Secretos. Crear un archivo `app-config.yaml` que defina desacopladamente los parámetros del sistema:

```
apiVersion: v1
kind: Namespace
metadata:
  name: produccion
---
apiVersion: v1
kind: ConfigMap
metadata:
  name: backend-config
  namespace: produccion
data:
  APP_ENV: "production"
  LOG_LEVEL: "info"
  DATABASE_HOST: "database-service.produccion.svc.cluster.local"
  DATABASE_PORT: "5432"
---
apiVersion: v1
kind: Secret
metadata:
  name: backend-secrets
  namespace: produccion
type: Opaque
stringData:
  DB_PASSWORD: "SuperSecureK8sPassword2026!"
  API_KEY: "icesi-ingesoft5-jwt-secret-token"
```

Tarea 2.2: Despliegue de la Capa de Datos (PostgreSQL) con Selección de Nodo. Garantizar que la base de datos se ejecute estrictamente en la máquina física **PC 5** (`tier=database`):

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: database-deployment
  namespace: produccion
spec:
  replicas: 1
  selector:
    matchLabels:
      app: postgres-db
  template:
    metadata:
      labels:
        app: postgres-db
    spec:
      nodeSelector:
        tier: database
      containers:
      - name: postgres
        image: postgres:15-alpine
        env:
        - name: POSTGRES_PASSWORD
          valueFrom:
            secretKeyRef:
              name: backend-secrets
              key: DB_PASSWORD
        ports:
        - containerPort: 5432
---
```

```
apiVersion: v1
kind: Service
metadata:
  name: database-service
  namespace: produccion
spec:
  selector:
    app: postgres-db
  ports:
  - port: 5432
    targetPort: 5432
  type: ClusterIP
```

Tarea 2.3: Despliegue del Backend con Anti-Afinidad de Pods y Probes de Salud. El backend debe desplegarse con **4 réplicas**. Para evitar que todas las réplicas queden en el mismo computador físico, se debe forzar una regla de `podAntiAffinity` para que se distribuyan de forma equitativa entre **PC 2** y **PC 3**:

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: backend-deployment
  namespace: produccion
spec:
  replicas: 4
  selector:
    matchLabels:
      app: backend-api
  template:
    metadata:
      labels:
        app: backend-api
    spec:
      affinity:
        nodeAffinity:
          requiredDuringSchedulingIgnoredDuringExecution:
            nodeSelectorTerms:
            - matchExpressions:
              - key: tier
                operator: In
                values: ["backend"]
        podAntiAffinity:
          preferredDuringSchedulingIgnoredDuringExecution:
          - weight: 100
            podAffinityTerm:
              labelSelector:
                matchExpressions:
                - key: app
                  operator: In
                  values: ["backend-api"]
              topologyKey: "kubernetes.io/hostname"
      containers:
      - name: api
        image: nginx:alpine
        resources:
          requests:
            cpu: "100m"
            memory: "128Mi"
          limits:
            cpu: "500m"
```

```
            memory: "256Mi"
        livenessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 10
          periodSeconds: 5
        readinessProbe:
          httpGet:
            path: /
            port: 80
          initialDelaySeconds: 5
          periodSeconds: 3
        ports:
        - containerPort: 80
---
apiVersion: v1
kind: Service
metadata:
  name: backend-service
  namespace: produccion
spec:
  selector:
    app: backend-api
  ports:
  - port: 8080
    targetPort: 80
  type: ClusterIP
```

Tarea 2.4: Despliegue del Frontend Web en PC 4. Desplegar el frontend (2 réplicas) asignado mediante `nodeSelector: tier: frontend` y exponer su servicio interno mediante `ClusterIP`.

2.3 Fase 3: Exposición de Tráfico y Enrutamiento con Ingress (PC 6)

El computador **PC 6** operará como la puerta de enlace perimetral (*Edge Ingress Gateway*).

1. Instalar o habilitar el Ingress Controller (en k3s viene Traefik activo por defecto, o desplegar NGINX Ingress Controller en el nodo etiquetado como `role=ingress-gateway`).

2. Crear el manifiesto de Ingress `app-ingress.yaml` para enrutar el tráfico HTTP de acuerdo al path solicitado:

```
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: cluster-ingress
  namespace: produccion
spec:
  rules:
  - host: ingesoft5.lab.icesi
    http:
      paths:
      - path: /api
        pathType: Prefix
        backend:
          service:
            name: backend-service
            port:
              number: 8080
      - path: /
        pathType: Prefix
```

```
        backend:
          service:
            name: frontend-service
            port:
              number: 80
```

3. En el computador **PC 7** (Bastion), configurar `/etc/hosts` para asociar `ingesoft5.lab.icesi` con la IP física de **PC 6**.

4. Verificar el acceso ejecutando consultas con `curl -I http://ingesoft5.lab.icesi/api` y certificar que responda código HTTP 200/404 esperado con balanceo hacia los Pods.

2.4 Fase 4: Pruebas de Resiliencia, Auto-reparación (Self-Healing) y Caos Controlado

En esta fase el equipo demostrará el verdadero poder de Kubernetes al someter el clúster a fallos intencionales. El estudiante a cargo de **PC 7 (Bastion/SRE)** coordinará la inyección de fallos y la medición del impacto.

Experimento 4.1: Fallo a nivel de proceso / contenedor (Pod Self-Healing).

1. Monitorear los pods de backend en vivo desde PC 7: `kubectl get pods -n produccion -w -o wide`.

2. En PC 1, forzar la eliminación abrupta de uno de los Pods de backend: `kubectl delete pod <nombre-pod>-n produccion --grace-period=0 --force`.

3. Observar y documentar cómo el `ReplicaSetController` detecta la discrepancia entre el estado deseado (4 réplicas) y el observado (3 réplicas), aprovisionando un nuevo Pod en menos de 2 segundos.

Experimento 4.2: Fallo de Hardware Físico (Simulación de Caída de Servidor).

Este experimento es el núcleo del laboratorio de 7 máquinas:

1. Iniciar un generador de tráfico constante desde **PC 7** apuntando al Ingress (PC 6):

```
# Ejecutar 5.000 peticiones continuas midiendo tasa de exito y errores
hey -n 5000 -c 10 -q 50 http://ingesoft5.lab.icesi/api
```

2. Durante la ráfaga de tráfico, **simular la caída del nodo PC 2 (**`k8s-worker-backend-1`**)**. Al operar sin `sudo`, el estudiante puede:

   - Desconectar físicamente el cable de red Ethernet del equipo PC 2, **o bien**:
   - Detener el contenedor del agente K3s directamente desde el usuario docker: `docker stop k8s-worker`.

3. **Observación del Control Plane:** Registrar en un cronómetro:

   - Momento en que el nodo pasa de `Ready` a `NotReady` en `kubectl get nodes`.
   - Acción del `kube-scheduler` al reprogramar las réplicas huérfanas hacia la máquina física **PC 3** (`k8s-worker-backend-2`).
   - Porcentaje de éxito de las peticiones en el reporte de `hey` (se debe certificar un éxito $\geq 99{,}5\,\%$ gracias al balanceo de carga automático y a las réplicas activas en PC 3).

4. Volver a conectar la red o reiniciar el contenedor (`docker start k8s-worker`) en PC 2 y documentar la reintegración automática del nodo al clúster sin intervención manual.

2.5 Fase 5: Actualización Progresiva (Rolling Update) y Rollback Inmediato

1. Actualizar la versión de la imagen en el Deployment de backend:

```
kubectl set image deployment/backend-deployment api=nginx:1.25.4-alpine -n produccion --record
```

2. Seguir la progresión del despliegue en tiempo real: `kubectl rollout status deployment/backend-deployment -n produccion`. Verificar que en ningún momento el número de réplicas disponibles cayó por debajo del umbral mínimo (`maxUnavailable: 0`).

3. Simular un despliegue defectuoso introduciendo una imagen con un tag inexistente (`nginx:tag-inexistente-error`).

4. Comprobar el estado `ImagePullBackOff` en los pods nuevos sin que los pods antiguos se detengan.

5. Ejecutar el comando de reversión inmediata: `kubectl rollout undo deployment/backend-deployment -n produccion` y verificar la restauración del servicio.

3 Cuestionario de Análisis Conceptual e Inteligencia Artificial Generativa

1. **Monolito Local vs Clúster Distribuido Real:** ¿Por qué las pruebas de resiliencia y auto-reparación ejecutadas sobre herramientas mononodo como Minikube o Docker Desktop enmascaran fallos críticos de red (latencias de CNI, quórum de etcd y particiones de red split-brain) que solo se hacen evidentes al operar sobre 7 máquinas físicas independientes?

2. **El Teorema CAP y etcd en el Control Plane:** Explique el funcionamiento del algoritmo de consenso Raft utilizado por `etcd`. Si en un clúster de alta disponibilidad se cuenta con 3 nodos de control plane, ¿cuántos nodos pueden fallar simultáneamente sin perder la capacidad de escritura? ¿Qué ocurre con los pods ya en ejecución si el plano de control se vuelve inaccesible?

3. **Balanceo de Carga L4 vs L7 (Services vs Ingress):** Detalle la diferencia entre el funcionamiento de `kube-proxy` (manejando tablas de `iptables` o `IPVS` a nivel de capa de transporte TCP/UDP) y un Ingress Controller basado en NGINX/Envoy operando en Capa 7 (HTTP/HTTPS con inspección de headers y URLs).

4. **Afinidad y Alta Disponibilidad Anti-SPOF:** En arquitecturas cloud-native, ¿por qué es mandatorio configurar `podAntiAffinity` con la clave `topologyKey: kubernetes.io/hostname` en servicios de misión crítica, y cómo interactúa esto con los `PodDisruptionBudgets` (PDB) durante ventanas de mantenimiento?

5. **Auditoría Crítica de Manifiestos K8s generados por IA:** Al solicitar a modelos de lenguaje (LLMs como GPT o Claude) la generación de manifiestos YAML para Kubernetes, identifique las tres alucinaciones o errores técnicos más recurrentes (ej. uso de `apiVersion` obsoletas como `extensions/v1beta1`, confusión entre `port`, `targetPort` y `nodePort`, omisión de probes o límites de memoria provocando OOMKills) y documente la estrategia empleada por el equipo para auditar y corregir el código.

4 Rúbrica de Evaluación Modularizada por Punto (Escala 0.0 a 5.0)

Criterio / Entregable	Pts	Excelente (4.5 – 5.0)	Aceptable (3.0 – 4.4)	Insuficiente (0.0 – 2.9)
Criterio 1: Topología y Clúster de 7 Nodos	**1.0**	Aprovisiona e integra con éxito las 7 máquinas en el clúster. Todos los nodos figuran en estado `Ready`. Aplica etiquetas semánticas y documenta la topología de red con claridad.	Logra unir entre 4 y 5 nodos al clúster, pero presenta inconsistencias en la resolución DNS local de hostnames o errores menores de swap en algún worker.	El clúster no supera 3 nodos, el Control Plane falla repetidamente o no logran conectividad entre las máquinas del grupo.
Criterio 2: Despliegue Declarativo y Afinidades	**1.0**	Define manifiestos limpios con separación de Namespaces, ConfigMaps y Secrets. Implementa reglas de `podAntiAffinity` distribuyendo las 4 réplicas equitativamente entre PC 2 y PC 3.	Despliega la aplicación pero no implementa anti-afinidad (quedando los pods en un solo worker) o expone contraseñas en texto plano sin usar Secrets.	Manifiestos con errores sintácticos de YAML, pods en estado `CrashLoopBackOff` continuo o falta de comunicación con la base de datos.
Criterio 3: Enrutamiento e Ingress L7	**1.0**	Configura el Ingress Controller en el nodo asignado (PC 6). Enruta correctamente el tráfico hacia frontend y backend mediante paths. Resuelve peticiones desde PC 7.	El Ingress funciona de manera intermitente, presenta errores en la reescritura de paths (`rewrite-target`) o solo accede vía IP directa sin nombre de host.	No logran configurar el Ingress Controller, o dependen exclusivamente de NodePorts sin balanceo perimetral.
Criterio 4: Resiliencia y Chaos Testing	**1.0**	Ejecuta la prueba de caída física desconectando PC 2 o deteniendo su contenedor durante la ráfaga de tráfico. Demuestra la reprogramación automática de pods y mide la tasa de éxito ($\geq$ $99{,}5\,\%$) sin caída global.	Realiza la desconexión pero no mide el impacto con herramientas de tráfico, o la aplicación sufre una interrupción prolongada por falta de réplicas en otros nodos.	No realizan la prueba de desconexión física de la máquina o el clúster colapsa por completo al apagar el worker.
Criterio 5: Cuestionario y Auditoría IAG	**1.0**	Responde las 5 preguntas conceptuales con solvencia técnica rigurosa. Documenta con evidencias los errores y alucinaciones corregidos en los manifiestos generados con IA.	Responde el cuestionario de manera teórica superficial sin contrastarlo con los datos observados en el laboratorio o con poca auditoría de IA.	Respuestas copiadas textualmente de internet sin análisis, incompletas o con errores conceptuales graves sobre K8s y etcd.
CALIFICACIÓN TOTAL	**5.0**	**Escala Institucional Universidad Icesi: Aprobatoria $\geq$ 3.0 • Duración: 3 Horas Lab**		

INSTRUCCIONES DE ENTREGA Y POLÍTICAS DE LABORATORIO

MODALIDAD ACADÉMICA

1. **Repositorio Git del Equipo:** El equipo debe crear un repositorio en GitHub / Bitbucket titulado `ingesoft5-taller-k8s-grupoXX` que contenga la carpeta `k8s-manifests/` con todos los archivos YAML organizados y un archivo `README.md` técnico.
2. **Evidencias en Video o Demostración Presencial:** Documentar en capturas de pantalla fechadas o enlace a video demostrativo de máximo 4 minutos:
   - Salida de `kubectl get nodes -o wide` con los 6 nodos `Ready`.
   - Salida de `kubectl get pods -A -o wide` mostrando los pods distribuidos entre PC 2, PC 3, PC 4, PC 5 y PC 6.
   - Momento de la desconexión física de PC 2 y la respuesta del comando `hey` desde PC 7.
   - Demostración del Rolling Update y Rollback exitoso.
3. **Entrega en Plataforma:** Un integrante del equipo subirá el enlace al repositorio y el informe técnico consolidado en PDF antes del plazo de entrega fijado por el docente.

LIMPIEZA Y RESTAURACIÓN DE LA ESTACIÓN (POST-LABORATORIO)

Al finalizar la sesión, cada estudiante debe dejar la máquina en su estado original sin alterar la configuración del laboratorio:

```
# En PC 1 (Master) o PC 2..6 (Workers):
docker rm -f k8s-master 2>/dev/null || docker rm -f k8s-worker 2>/dev/null
rm -rf ~/k8s-cluster
```